WEEK – 2 (Skill)

Name: Simran Jangid

Rollno: 2520030166

Section: 8

Experiment No. 4(Q4)

Title: Command Tokenization and Parser Design using C

Aim: To implement a basic command parser that splits user input into tokens, identifies delimiters, handles whitespace, validates the token stream, detects syntax errors and empty commands, and produces a structured representation of the command for execution.

Objective:

- To split the input command into tokens.
- To identify delimiters such as |, ;, <, and >.
- To handle spaces and extra whitespace correctly.
- To create token structures for commands and arguments.
- To validate the generated token stream.
- To debug and display parsing output.
- To design basic parser logic.
- To generate a simple parse structure.
- To validate command syntax.
- To detect invalid or missing commands.
- To handle empty commands.
- To produce an execution structure.

Theory:

1. Tokenization

Tokenization is the process of breaking a command entered by the user into smaller meaningful units called **tokens**.

For example:

ls -l | grep txt

can be divided into:

ls

-l

|

grep

txt

Each token can represent a command, argument, or delimiter.

---

2. Delimiters

Delimiters are special symbols that separate different parts of a command.

Common shell delimiters include:

Delimiter	Meaning
`	`
;	Separates commands
<	Input redirection
>	Output redirection

Example:

cat file.txt | grep hello

Here | is the delimiter between two commands.

---

3. Whitespace Handling

Spaces, tabs, and newlines can occur between tokens.

For example:

ls -l

should produce the same tokens as:

ls -l

Therefore, the parser should ignore unnecessary whitespace.

---

4. Token Structure

The parser can store every token together with its type.

Example:

TOKEN: ls TYPE: COMMAND

TOKEN: -l TYPE: ARGUMENT

TOKEN: | TYPE: PIPE

TOKEN: grep TYPE: COMMAND

TOKEN: hello TYPE: ARGUMENT

---

5. Token Stream Validation

After tokenization, the parser checks whether the tokens are arranged correctly.

For example:

ls | grep hello

is valid.

But:

ls | | grep

is invalid because two pipe delimiters occur consecutively.

Similarly:

| ls

is invalid because the command starts with a pipe.

---

6. Parser Logic

The parser processes the token stream and determines how the command should be executed.

Example:

ls -l | grep txt

can be represented as:

```
    PIPE
   /   \
  ls    grep
```

| |

-l txt

This represents the relationship between the commands.

---

7. Empty Commands

The parser should detect empty input.

Example:

Input:

Output:

Error: Empty command

It should also detect cases such as:

ls |

because the pipe requires another command after it.

Algorithm:

- Start the program.
- Read a command from the user.
- Check whether the input is empty.
- Remove unnecessary leading and trailing whitespace.
- Scan the input character by character.
- Identify words separated by whitespace.
- Identify special delimiters such as |, ;, <, and >.
- Store each item as a token.
- Assign a type to each token.
- Display the generated token stream.
- Validate the token sequence.
- Detect invalid syntax such as consecutive pipes.
- Detect commands beginning or ending with invalid delimiters.
- Generate a simple parse/execution structure.
- Display parsing results.
- Stop.

Functions Used:

Function	Purpose
main()	Starting point of the program
printf()	Displays messages and output
fgets()	Reads the command entered by the user
strcspn()	Removes the newline \n from input
isspace()	Checks for spaces, tabs, and whitespace
isDelimiter()	Checks whether a character is `
printType()	Displays the type of each token
strlen()	String-length function from <string.h>
return	Terminates the function/program

Sample Output:

Enter command: ls -l | grep txt

--- Token Stream ---

Token 1: ls Type: COMMAND

Token 2: -l Type: ARGUMENT

Token 3: | Type: DELIMITER

Token 4: grep Type: ARGUMENT

Token 5: txt Type: ARGUMENT

--- Syntax Validation ---

Syntax is valid.

--- Execution Structure ---

ls -l [|]

grep txt

Observation:

The program successfully reads the user-entered command, separates it into individual tokens, identifies delimiters, handles whitespace, and displays the token type. It also validates the token stream and detects errors such as empty commands, consecutive delimiters, and commands starting or ending with delimiters.

Result:

The command tokenization and parser logic were successfully implemented. The program correctly generated the token structure, validated the syntax, detected parsing errors, handled empty commands, and produced the corresponding execution structure.

Part 2: Design and Implementation of Parser Logic for Command Syntax Validation and Execution Structures

Aim: To design parser logic for tokenized commands, generate parse trees, validate syntax, detect errors and empty commands, and produce appropriate execution structures.

Commands / Functions Used

1. gcc
gcc is used to compile the C source code into an executable program. It helps verify that the parser implementation has no compilation errors.

2. ./program_name
This command is used to execute the compiled parser program in Linux. It allows the user to enter commands and observe tokenization, syntax validation, error detection, and execution structure.

3. printf()
printf() displays the parse tree, syntax-validation messages, errors, and execution structure. It is also useful for debugging the parser output.

4. fgets()
fgets() reads the complete command entered by the user, including spaces. It is useful for accepting commands that contain multiple tokens.

5. isspace()
isspace() checks whether a character is whitespace such as a space or tab. It helps the parser ignore unnecessary whitespace while processing input.

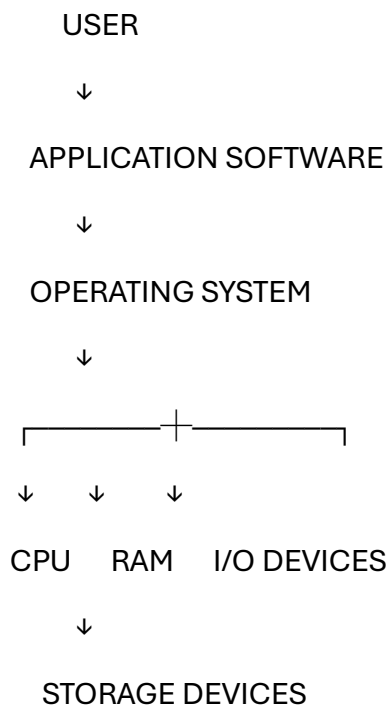
6. isDelimiter() *(user-defined function)*
This function identifies delimiters such as |, ;, <, and >. It helps the parser separate commands and arguments correctly.

Hardware and Operating System Relationship

Hardware refers to the physical components of a computer, such as the **CPU, RAM, hard disk/SSD, keyboard, mouse, and display**.

The **Operating System (OS)** acts as an interface between the user/application software and hardware. It manages hardware resources and provides services to programs.

Relationship



How OS interacts with hardware

1. **CPU Management** – OS schedules processes and allocates CPU time.
2. **Memory Management** – OS allocates and manages RAM for programs.
3. **File Management** – OS manages data stored on HDD/SSD.
4. **Device Management** – OS controls devices such as keyboard, mouse, printer, and display using device drivers.

5. **Process Management** – OS creates, executes, and terminates processes.
6. **Security** – OS controls access to hardware and system resources.

Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define MAX_INPUT 200
#define MAX_TOKENS 50
typedef enum {
  COMMAND,
  ARGUMENT,
  DELIMITER
} TokenType;
typedef struct {
  char value[50];
  TokenType type;
} Token;
void printType(TokenType type) {
  if (type == COMMAND)
    printf("COMMAND");
  else if (type == ARGUMENT)
    printf("ARGUMENT");
  else
    printf("DELIMITER");
}
int isDelimiter(char ch) {
  return ch == '|' || ch == ';' || ch == '<' || ch == '>';
```

```
}
int main() {
    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];
    int count = 0;
    int i = 0;
    int firstWord = 1;
    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);
    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';
    /* Check empty command */
    i = 0;
    while (input[i] != '\0' && isspace(input[i]))
        i++;
    if (input[i] == '\0') {
        printf("Error: Empty command.\n");
        return 0;
    }
    /* Tokenization */
    i = 0;
    while (input[i] != '\0' && count < MAX_TOKENS) {
        /* Ignore whitespace */
        if (isspace(input[i])) {
            i++;
            continue;
        }
        /* Delimiter */
```

```
    if (isDelimiter(input[i])) {
        tokens[count].value[0] = input[i];
        tokens[count].value[1] = '\0';
        tokens[count].type = DELIMITER;
        count++;
        i++;
        continue;
    }
    /* Word / command / argument */
    int j = 0;
    while (input[i] != '\0' &&
           !isspace(input[i]) &&
           !isDelimiter(input[i])) {
        if (j < 49)
            tokens[count].value[j++] = input[i];
        i++;
    }
    tokens[count].value[j] = '\0';
    if (firstWord) {
        tokens[count].type = COMMAND;
        firstWord = 0;
    } else {
        tokens[count].type = ARGUMENT;
    }
    count++;
}
/* Display tokens */
printf("\n--- Token Stream ---\n");
```

```
for (i = 0; i < count; i++) {
   printf("Token %d: %-10s Type: ",
        i + 1, tokens[i].value);
   printType(tokens[i].type);
   printf("\n");
}
/* Syntax validation */
printf("\n--- Syntax Validation ---\n");
int error = 0;
if (tokens[0].type == DELIMITER) {
   printf("Error: Command cannot start with a delimiter.\n");
   error = 1;
}
for (i = 0; i < count - 1; i++) {
   if (tokens[i].type == DELIMITER &&
      tokens[i + 1].type == DELIMITER) {
      printf("Error: Consecutive delimiters found: %s %s\n",
           tokens[i].value,
           tokens[i + 1].value);
      error = 1;
   }
}
if (tokens[count - 1].type == DELIMITER) {
   printf("Error: Command cannot end with a delimiter.\n");
   error = 1;
}
if (!error) {
   printf("Syntax is valid.\n");
```

```
        printf("\n--- Execution Structure ---\n");
        for (i = 0; i < count; i++) {
            if (tokens[i].type == DELIMITER)
                printf("[ %s ]\n", tokens[i].value);
            else
                printf("%s ", tokens[i].value);
        }
        printf("\n");
    }
    return 0;
}
```

OUPUT:

```
teja@Hasini:~$ nano parser.c
teja@Hasini:~$ gcc parser.c -o parser
teja@Hasini:~$ ./parser
Enter command: ls -l | grep txt

--- Token Stream ---
Token 1: ls          Type: COMMAND
Token 2: -l          Type: ARGUMENT
Token 3: |           Type: DELIMITER
Token 4: grep        Type: ARGUMENT
Token 5: txt         Type: ARGUMENT

--- Syntax Validation ---
Syntax is valid.

--- Execution Structure ---
ls -l [ | ]
grep txt
teja@Hasini:~$ ./parser
Enter command:
Error: Empty command.
teja@Hasini:~$ ./parser
Enter command: ls | | grep

--- Token Stream ---
Token 1: ls          Type: COMMAND
Token 2: |           Type: DELIMITER
Token 3: |           Type: DELIMITER
Token 4: grep        Type: ARGUMENT

--- Syntax Validation ---
Error: Consecutive delimiters found: | |
```

```
teja@Hasini:~$ ./parser
Enter command: | ls

--- Token Stream ---
Token 1: |           Type: DELIMITER
Token 2: ls          Type: COMMAND

--- Syntax Validation ---
Error: Command cannot start with a delimiter.
teja@Hasini:~$ ls
Experiment-2  linux  os_task  os_task.c  parser  parser.c
teja@Hasini:~$
```